

4. Change to the *contacts* directory. Check if the setup is proper, by running the basic application, as follows:

```
npm start
```

This will start the application in development mode and 'Welcome to the app' will be displayed.

5. Install the modules for Redux integration, as follows:

```
npm install -S redux
npm install -S redux-thunk
npm install -S @angular-redux/store
```

6. Create the directory, *client*, under *contacts* and move the entire folder structure from *contacts* to *client* so that the *client* folder contains the code for the client-side Angular app.
7. To proxy API requests during development and avoid CORS issues, modify the 'start' script in *package.json* present in the *client* folder with the following command:

```
"start": "ng serve --proxy-config proxy.conf.json"
```

8. Create the file *proxy.conf.json* under the *client* folder with the following content:

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false
  }
}
```

9. Create a directory *server* under *contacts*. From this folder, execute *npm init*, and install the *express* and *body-parser* module to serve the persistent data required for the *Contacts* application.

```
npm init -y
npm install -S express
npm install -S body-parser
```

10. Add the 'start' setting under 'scripts' in *package.json* present in the *server* folder with the following command so that we can run the Express server application using 'npm start'.

```
"start": "node server.js"
```

Now we can start working on adding the features required for the *Contacts* application. While explaining the implementation, the main code snippets have been provided. For the complete code, please refer to the GitHub repository at <https://github.com/srini-wip/angular-redux-contacts.git>.

Serving the data required by the *Contacts* app using Express

Let's now implement the code to serve the data required by the *Contacts* application using the Express Web framework.

Create a file *server.js* under the folder *contacts*. To provide the list of contacts, implement the API endpoint */api/contacts* using the GET method. This will read the JSON file, *contacts.json*, for the list of available contacts and send the response as JSON data.

```
app.get('/api/contacts', function(req, res) {
  // read from JSON file and send the response as JSON data
});
```

To add a contact to the contact list, implement the API endpoint */api/contacts* using the POST method. To retrieve the data sent by the browser, use *body-parser* middleware, which inserts the required data in the request object, from which we can easily extract and save it to the JSON file.

```
app.post('/api/contacts', function(req, res) {
  // extract the data and save to JSON file
});
```

To delete a contact from the contact list, implement the API endpoint */api/contacts/:id* using the DELETE method. This will delete the contact and update the JSON file.

```
app.delete('/api/contacts/:id', function(req, res) {
  // delete the contact and update the JSON file
});
```

Implementing the user interface using Angular and Redux

Let's now implement the set of features on the client side using Angular and Redux.

When generating the new project, Angular CLI would have created a file *app.module.ts*, which is the root module and contains the declaration of libraries and components.

Configuring the store

In the constructor of the *class AppModule*, we need to configure the store. We need to provide the *rootReducer*, initial state (if any), middlewares and then a store enhancer. Since we will be using the *redux-thunk* middlewares to implement asynchronous actions, the third parameter will be the *thunk* middlewares. We need to include the necessary imports for Redux, *redux-thunk* and also specify the *NgReduxModule* in the imports section of the *@NgModule* decorator for *AppModule*.

```
constructor (ngRedux: NgRedux<any>) {
  ngRedux.configureStore(rootReducer, {}, [ thunk ], []);
}
```